WEEK 4

Agentic Code Review and Testing

Verification and quality gates for AI-generated code



References

01 O'Reilly, *AI-Native Software Engineering* (early release), Ch. 8 "Verification and Quality Gates."

02 E. Goldratt, *The Goal*, North River Press, 1984. Theory of Constraints

03 B. Böckeler, "Harness Engineering for Coding-Agent Users." martinfowler.com

04 Faros AI, *AI Engineering Report 2026: The Acceleration Whiplash*.

05 GitHub (Aug 20, 2026); Factory AI, Linear and Uber figures reported by The Pragmatic Engineer.

06 M. Sharma et al., "Towards Understanding Sycophancy in Language Models," 2023. arXiv:2310.13548

07 Meta Engineering, "LLMs Are the Key to Mutation Testing," 2025. engineering.fb.com

08 Google, "Small CLs" and SRE Workbook, "Alerting on SLOs."

09 OWASP Top 10 and Top 10 for LLM Applications.

10 A. Osmani (2026), on verification as the new bottleneck.

What we covered so far

The first three stages of the SDLC, with agents in the loop

01 · DONE

Planning

Specs pin down intent before any code is written.

02 · DONE

Design

Conventions and architecture decisions the agent must follow.

03 · DONE

Implementation

Coding agents, the harness and the context they work in.

04 · TODAY

Testing

Review and verification of what the agent produced.

TODAY **The agent wrote the code. How do we know it is correct?**

Where we are in the SDLC

The loop now cycles in hours. Today we stop at the Test stage.



SO FAR · INPUTS

How do you guide the agent toward the right answer?

TODAY · FEEDBACK

How do you know the code it wrote is correct, with 20 more PRs waiting?

Today

00
Why verification matters

01
The verification bottleneck

02
Deterministic guardrails

03
LLM-based review

04
Safe deployment strategies

05
Runtime safety and AI-powered ops

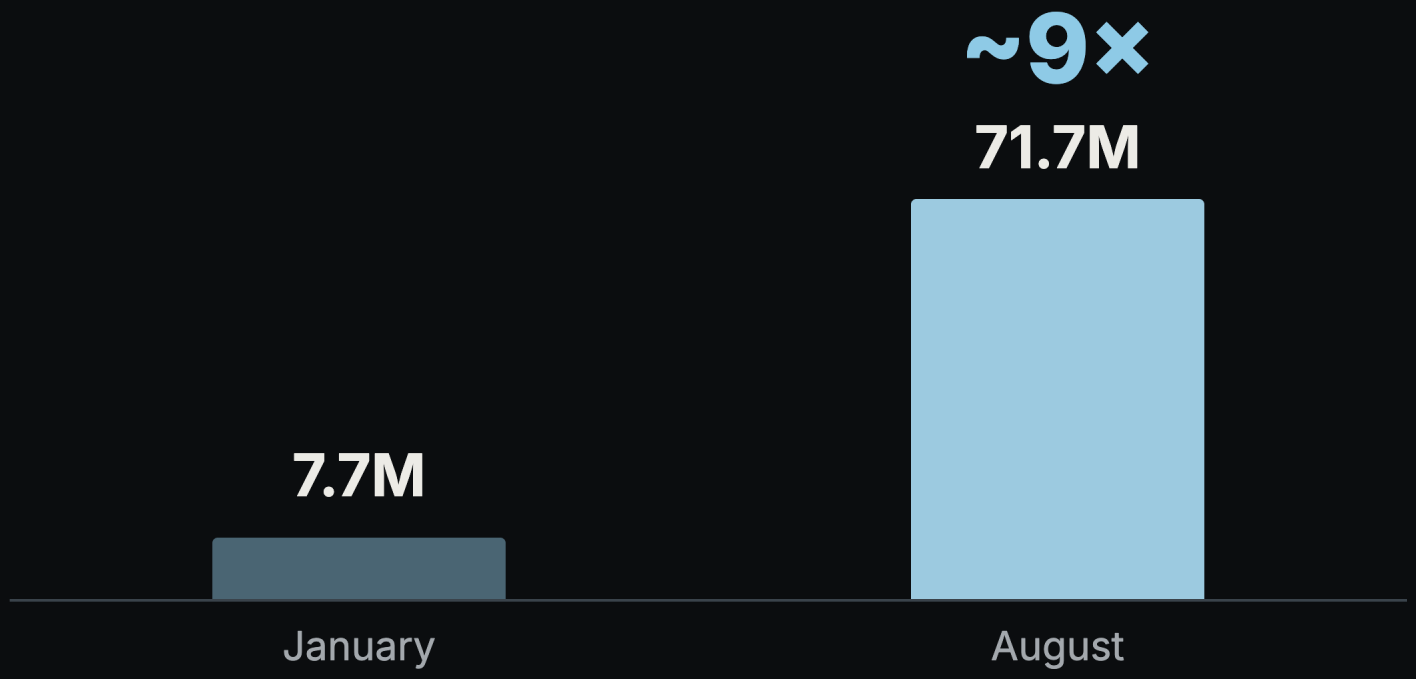
06
The human in the loop

07
Putting it together

WHY VERIFICATION MATTERS

Agent-generated work is growing fast

Agent-authored PRs on GitHub, January to August 2026



ALL GITHUB ACTIVITY · AUG 2026

130M
merged PRs per month

2.9B
commits per month

24M
new repositories per month

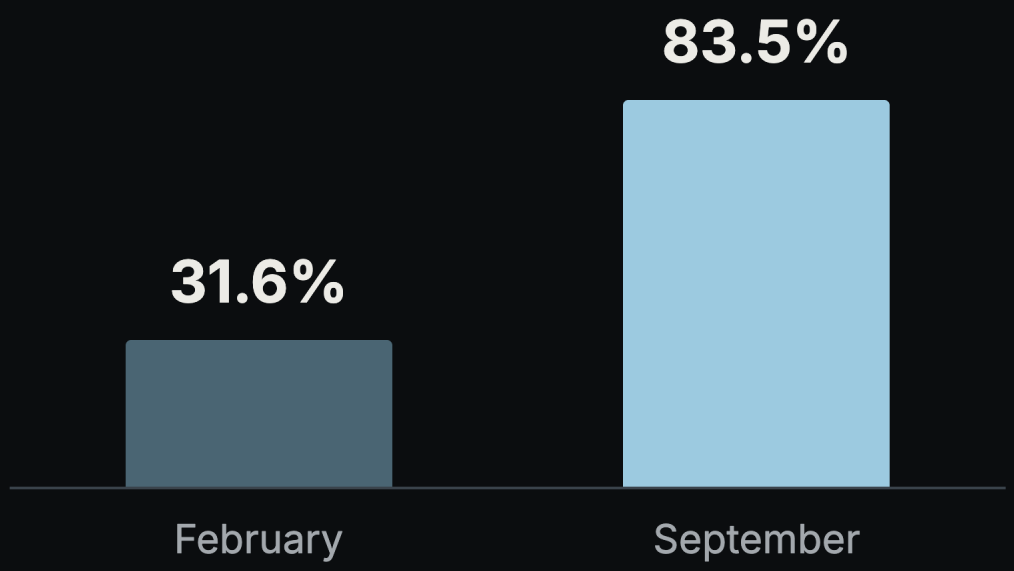
More changes create more work to verify.

Sources: GitHub (August 20, 2026) · Agent-authored PR figures reported by The Pragmatic Engineer

Agent workflows are becoming routine

Examples from Factory AI, Linear and Uber in 2026

FACTORY AI · USERS USING SKILLS



UBER · FEBRUARY TO AUGUST

7×
weekly active users

9.4×
agent requests

Overall cost levels off as usage grows.

Linear reports more agent-created than human-created issues from July 2026.

Sources: Factory AI, Linear and Uber · Figures reported by The Pragmatic Engineer

WHY VERIFICATION MATTERS

AI adoption brings real delivery gains

Faros: 22,000 developers, more than 4,000 teams, two years of telemetry

+16.2%

PR merges

per developer

+33.7%

Task throughput

per developer

+66%

Epics completed

per developer

Within-organization comparison: periods of lowest versus highest AI adoption

Source: Faros AI, AI Engineering Report 2026: The Acceleration Whiplash

WHY VERIFICATION MATTERS

Quality costs rise alongside throughput

Faros low-to-high AI adoption comparisons

+861%

Code churn

Deleted-to-added lines ratio for merged code

+242.7%

Incidents per PR

A ratio, not the probability that a PR causes an incident

+54%

Bugs per developer

The 2025 report found +9%

Churn can reflect useful refactoring as well as rework.

Source: Faros AI (2026) · Observational telemetry; these are reported increases and do not establish causation

WHY VERIFICATION MATTERS

Review capacity is under pressure

Faros: review delays and unreviewed merges increase with AI adoption

+441.5%

Median time in review

+156.6%

Median time to first review

+199.6%

Average time spent in review

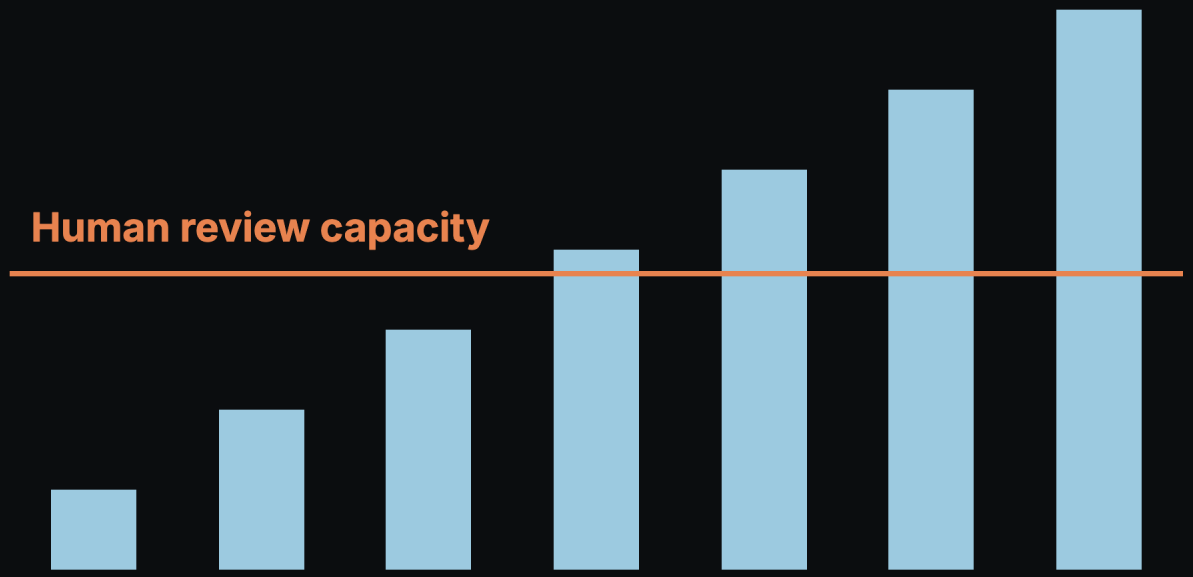
+31.3% **PRs merged without any human or agent review**

Source: Faros AI (2026) · This motivates the review and testing methods that follow

01 The verification bottleneck

The bottleneck has moved

Faster generation increases pressure on downstream verification.



AI reduces the cost of producing code

Reading speed and accountability do not scale at the same rate

The constraint shifts to verification

Conceptual illustration · Theory of Constraints · Osmani (2026)

Why manual review doesn't scale

The same team, the month it picks up AI tools

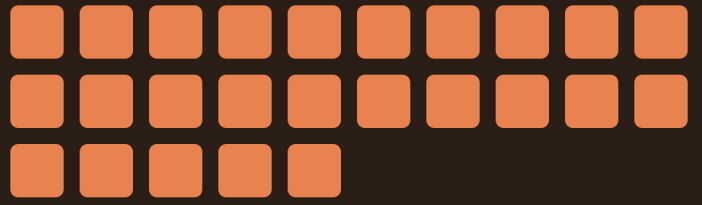
5 PRS A DAY



~1 hour

per review. The reviewer reads the diff, thinks through edge cases, leaves comments that mean something.

25 PRS A DAY



~5 min · "LGTM"

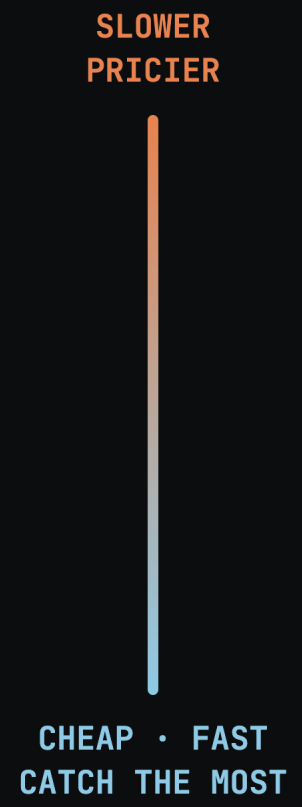
Approvals get faster, comments shorter. Not laziness: the work cannot be done properly.

A rubber stamp is not a safety net. It is the illusion of one.

The verification stack

Five filters. Each layer assumes the ones below it are running.

L5	**Focused human review**	Intent, fit, is this the right thing to build?
L4	**Runtime safety**	Observability, anomaly detection, AI incident analysis
L3	**Deploy-time safety**	Feature flags, canaries, blue-green: limit blast radius
L2	**LLM-based review**	Architecture fit, spec alignment, security intent
L1	**Deterministic guardrails**	Linters, types, tests, scanners: hard yes or no



The two big questions

BEFORE MERGE

How do you know this code is safe to ship?

Which checks must pass first? Once the machines are done, what is left for a human to judge?

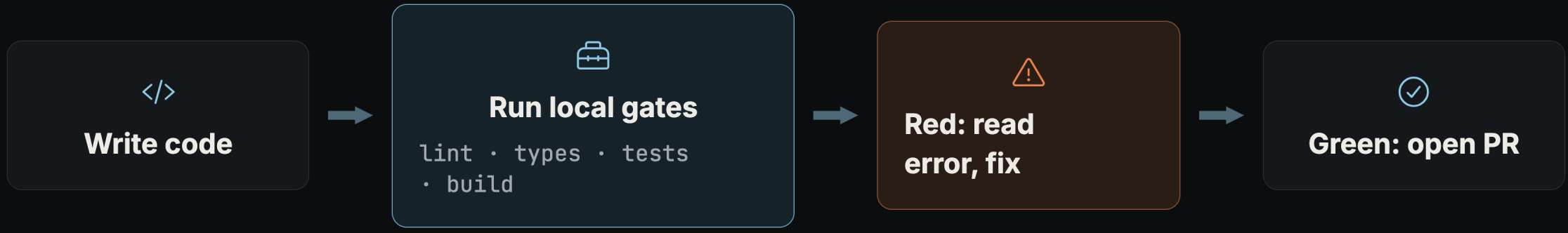
AFTER MERGE

How fast can you tell production has broken, and fix it?

Some bugs need real traffic, real scale or real timing to appear. Never once on your laptop.

Harness validations and self-check loops

The agent runs your gates before it ever opens a pull request.



PACKAGE.JSON

```
"scripts": {
  "ci": "eslint . && tsc --noEmit
       && vitest run && vite build"
}
```

Reason, act, **observe**, adjust: a failing test becomes an observation the agent can act on.

With a headless browser or MCP server, it can click through real user flows too.

Read: B. Böckeler, Harness engineering for coding-agent users · martinfowler.com

BEFORE MERGE · LAYER 1

02 Deterministic guardrails

Pick the checks your project needs

Each one is fast, automatic and gives the same answer every time.

CODE QUALITY

- Lint and format
- Dead code
- Type checks
- PR size

TESTS

- Test suite
- Mutation testing
- Property-based

SECURITY

- SAST
- Dependencies
- Secrets
- Infra as code

DESIGN AND USERS

- Architecture rules
- Performance budgets
- API contracts
- Accessibility

MATCH YOUR RISK Internal tool: secrets scanning and tests. Public API: contracts and dependency scanning.

Linting and formatting

WHY IT MATTERS FOR AI CODE Each agent session writes in a slightly different style.

FORMATTERS

One allowed layout. Nobody reviews indentation again.

Prettier · gofmt

LINTERS

Sweep up the long tail of small, real problems.

ESLint · Ruff · golangci-lint

WHAT A LINTER CATCHES

```
async function save(user) {
  const old = load(user.id)
  db.write(user)
}
```

```
× 'old' is assigned but never used
× Promise returned by db.write is not awaited
```

IN PRACTICE Commit the configs, run them in a pre-commit hook and in CI (formatter in check mode).

Dead code and unused dependencies

WHY IT MATTERS FOR AI CODE Agents abandon an approach and leave its code and packages behind.

```
$ npx knip
Unused files (2)
  src/parser/parserV1.ts
  src/utils/tryFetch.ts
Unused exports (1)
  formatDateLegacy src/date.ts
Unused dependencies (1)
  axios package.json
```

THE COST

Dead code never fails a test. It piles up until the next reader, human or agent, follows paths that lead nowhere.

IN PRACTICE

Clear the backlog once, then fail CI on new findings. Legacy repo: report-only first.

Knip (JS/TS) · vulture (Python)

Type checks and compilation

WHY IT MATTERS FOR AI CODE Models get the code's shape right but invent methods and fields.

TS2339

```
const user = await getUser(id)
return user.getFullName()
```

```
× Property 'getFullName' does not exist on type 'User'.
```

The model "remembered" a method that is not there. A strict check turns that into a red build in seconds.

TYPICAL HALLUCINATIONS

- A method that does not exist
- Three arguments where two fit
- A string where an object is expected
- An invented field on a type

TypeScript strict · mypy · pyright

The test suite

WHY IT MATTERS FOR AI CODE When one agent writes the code and its tests, “all green” is worth less.

PASSED

```
def test_apply_discount():
    cart = Cart([Item(100)])
    cart.apply_discount("SAVE10")
    assert cart is not None
```

Runs every line of the new code. Asserts nothing about the total.

COVERAGE THEATER

100%

line coverage. Zero behavior verified.

Ask “are they checking behavior?”, not “do they pass?”

Mutation testing as a test quality gate

WHY IT MATTERS FOR AI CODE Shows whether AI-written tests would notice if the code were wrong.

ORIGINAL

```
if age ⩾ 18:
```

→

MUTANT

```
if age > 18:
```

→

Killed: a test went red

Survived: tests stayed green

WHAT A SURVIVOR TELLS YOU

No test ever checked that logic. Nobody tested age = 18.

IN PRACTICE

Slow. Run on files the PR touches; hold a score on payments, auth, core logic.

Stryker (JS, C#, Scala) · PIT (Java) · mutmut (Python) · Meta: LLM-generated mutants

Property-based testing

WHY IT MATTERS FOR AI CODE Agents handle the cases you describe and miss the edge cases you don't.

EXAMPLE-BASED

```
assert sort([3,1,2]) == [1,2,3]
```

One input, the one you thought of.

PROPERTY-BASED · HYPOTHESIS

```
@given(st.lists(st.integers()))
def test_sort_idempotent(xs):
    once = sort(xs)
    assert sort(once) == once
```

Empty lists, negatives, duplicates, huge values.

GOOD PROPERTIES SOUND LIKE LAWS

- decode(encode(x)) == x
- Money is conserved in a transfer
- Running twice = running once

fast-check (JS) · Hypothesis (Python)

Static application security testing

WHY IT MATTERS FOR AI CODE Models learned from public code, and a lot of it is insecure.

FLAGGED · SQL INJECTION

```
q = "SELECT * FROM users
     WHERE name = '" + name + "'"
db.execute(q)
```

User input glued into the query string.

SAFE · PARAMETERIZED

```
db.execute(
  "SELECT * FROM users WHERE name = ?",
  (name,))
```

The database handles the value separately.

Also catches: unescaped output (XSS), unsafe deserialization, weak crypto, path traversal.

Semgrep · SonarQube · more SAST tools

Dependency and secrets scanning

WHY IT MATTERS FOR AI CODE Agents pick packages from memory and may hardcode keys to get unblocked.

DEPENDENCIES · CONTAINERS

Agents grab whatever package they remember: maybe stale

Fail on high or critical.

Trivy

SECRETS

```
AWS_KEY = "AKIA..."
# TODO move to env
```

Caught before commit and again in CI.

Gitleaks

INFRASTRUCTURE AS CODE

A public S3 bucket works in every demo. It is one line from a data breach.

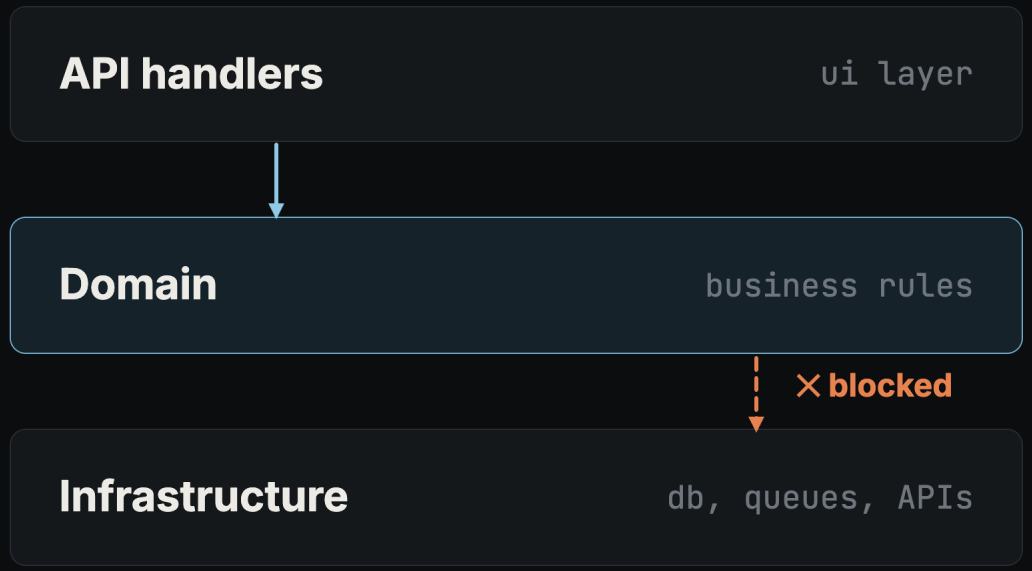
Scan Terraform and Kubernetes.

Checkov · Trivy IaC

Architecture fitness functions

Tests for structure, not behavior. The wiki rule becomes a failing build.

WHY IT MATTERS FOR AI CODE Agents take shortcuts across layers; hundreds of them erase the design.



EXAMPLE RULES

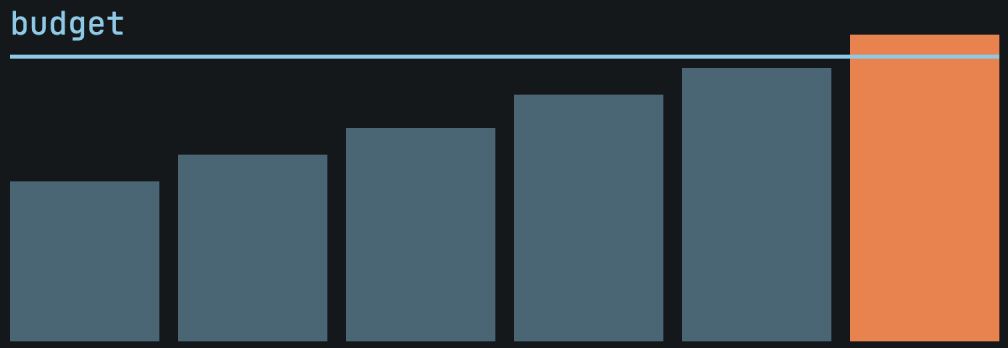
- Domain must not import infrastructure
- No circular module dependencies
- Service A never calls B directly

dependency-cruiser · ArchUnit · guide

Performance and query budgets

WHY IT MATTERS FOR AI CODE Agents add more than they delete and write the naive query by default.

BUNDLE SIZE CREEP



Each PR is “only 5 KB more.” The budget fails the one that crosses the line.

Lighthouse CI · backend benchmark gate

THE N+1 QUERY

```
for order in orders:  # 100 rows
    order.customer    # +1 query each
```

101 queries vs. **1 batched join**

Fine on 100 test rows, melts on 100 million. Gate on query count, slow queries and EXPLAIN plan diffs.

Accessibility and internationalization

WHY IT MATTERS FOR AI CODE AI frontend code often skips labels and alt text, and hardcodes English.

INACCESSIBLE BY DEFAULT

- Missing form labels
- Weak color contrast
- Keyboard traps
- Images with no alt text

axe in component or end-to-end tests

ENGLISH-ONLY BY DEFAULT

```
<button>Save changes</button>
<button>{t("save")}</button>
```

A lint rule for hardcoded user-facing strings

Pull-request size and scope discipline

WHY IT MATTERS FOR AI CODE Ask an agent for one feature and you may get 2,000 lines of changes.



THE RISK

The human is the last backstop, and no human meaningfully reviews 2,000 lines.

IN PRACTICE

A bot flags PRs over a threshold. Prompt agents for one change per PR.

Read: Google engineering practices, Small CLs

BEFORE MERGE · LAYER 2

03 LLM-based review

Why deterministic checks are not enough

DETERMINISTIC GATES SAY

The code is well-formed.

A hard yes or no.

AN LLM REVIEWER ASKS

Is it doing the right thing?

An opinion. It can be wrong.

THREE PLACES IT CAN RUN · MOST TEAMS USE MORE THAN ONE

Locally

Fresh session, before a PR exists

On the PR

A bot commenting where reviewers work

In the cloud

A managed service watching the repo

Adversarial review before the PR

The agent that just wrote the code is a terrible judge of it.

SAME SESSION

"Is this correct?"

"Yes, it handles all the cases."

It reasoned its way here two minutes ago, so it mostly agrees with itself: sycophancy.

FRESH SESSION · HOSTILE BRIEF

```
Find what is wrong with this change.
Assume it has bugs. Hunt for the
edge cases the author missed.
```

Only the diff and the context, no memory of writing it. Finds the unhandled null, the off-by-one.

RULE OF THUMB Use a different model to review: more like a colleague than proofreading your own writing.

How an LLM reviewer works

Roughly what a person does, at the speed of a bot.



EXAMPLE TOOLS · NOT FROM THE BOOK; PRODUCTS TURN OVER EVERY FEW MONTHS

CodeRabbit · GitHub Copilot code review · Claude Code GitHub Actions · PR-Agent (open source)

Chase the capability, not the product. The hard part is what to ask it and how to keep the noise down.

Two ways to organize the review

Start simple; specialize where it pays off.

TEAM OF SPECIALISTS



+ sharp focus · – cost and complexity

SINGLE REVIEWER + CHECKLIST · START HERE

```
pass  Follows repository layer?
fail  Input validated?
pass  Tests assert behavior?
n/a   Docs updated?
```

+ cheap, easy to tune · graduate to a specialist only where one area, usually security, deserves it

What to look for

Working code is the floor. The value is in questions past “does it run?”

CORRECTNESS AND FIT

Does the change fit?

Architecture: matches ADRs and existing patterns

Spec: every acceptance criterion, nothing extra

Tests: meaningful assertions and edge cases

RISK AND COMPLIANCE

Is it risky?

Security: OWASP Top 10, LLM Top 10

Privacy: new PII in a log or export

Cost: a fan-out query, a model call per page load

COMPLETENESS AND HYGIENE

Is it finished?

Docs: README, runbook and API docs match

Leftovers: debug logs, commented-out blocks, orphan helpers

LLM security review backs up SAST and data-flow tools; it is weak at tracing data across many files.

Tuning the reviewer to your codebase

```
# Role
Senior backend reviewer for our payments service.
Only raise issues that matter. Do not invent problems.
# Our conventions
- Money is integer cents, never floats.
- Every external call has a timeout and retry.
# Past incidents to watch for
- INC-204: missing idempotency key, double charge.
# Do not review
- /generated, /vendor, *.snap
# Output
file:line, blocker | warning | nit, one-line fix.
Verdict: APPROVE or REQUEST_CHANGES.
```

- **Concrete rules**, not fuzzy values
- **Real incidents** beat “check for bugs”
- **What to ignore** kills a class of noise
- **Fixed output** is quick to scan

Every wrong comment is an edit to this file.
Context engineering, aimed at the reviewer.

AFTER MERGE · LAYER 3

04 Safe deployment strategies

Feature flags as the default

Ship every nontrivial change behind a flag that defaults to off.

10:00 · DEPLOY

Code is live but dark. Doing nothing.

16:00 · FLAG ON

Released while you are watching.

16:20 · MISBEHAVES → FLAG OFF

Instant kill switch. No new deploy.

CHECKOUT.TS

```
if (flags.enabled("new-checkout")) {
  return newCheckout(cart)
}
return legacyCheckout(cart)
```

Deployment and release become two decisions.

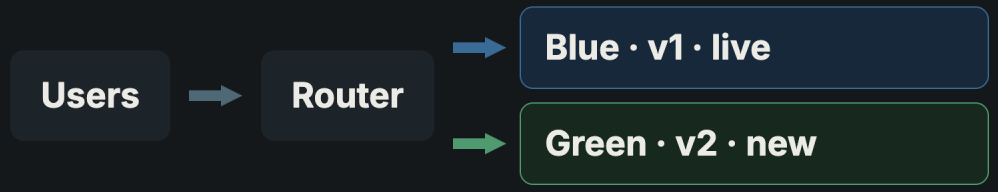
Watch for flag debt: stale flags nobody removes.

Blue-green and canary deployments

Two ways to move users from the old version (v1) to the new one (v2).

BLUE-GREEN · EVERYONE SWITCHES AT ONCE

Two copies of production. Users see only one.

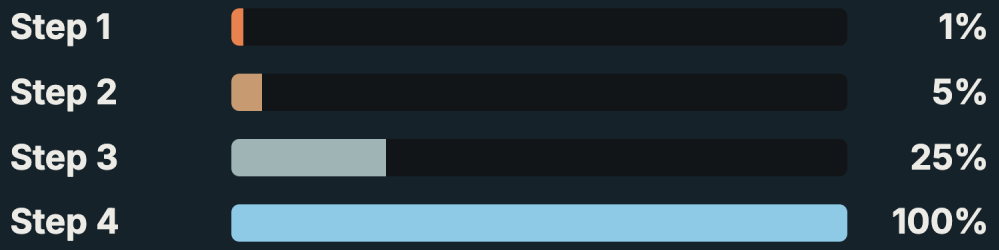


1. Deploy v2 to Green while Blue serves users
2. Test Green with no real users on it
3. Flip the router: all users go to Green
4. Problem? Flip back to Blue

Trade-off: instant way back, but a hidden bug reaches everyone at once.

CANARY · A FEW USERS FIRST

Send a small share of users to v2, then grow it while it stays healthy.

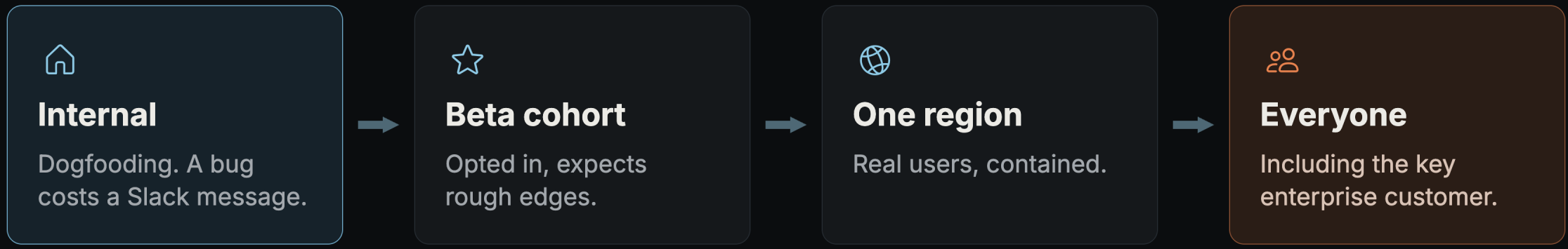


Before each step, check errors and speed. Problem? Send everyone back to v1.

Trade-off: slower rollout, but a bug only reaches a small slice of users.

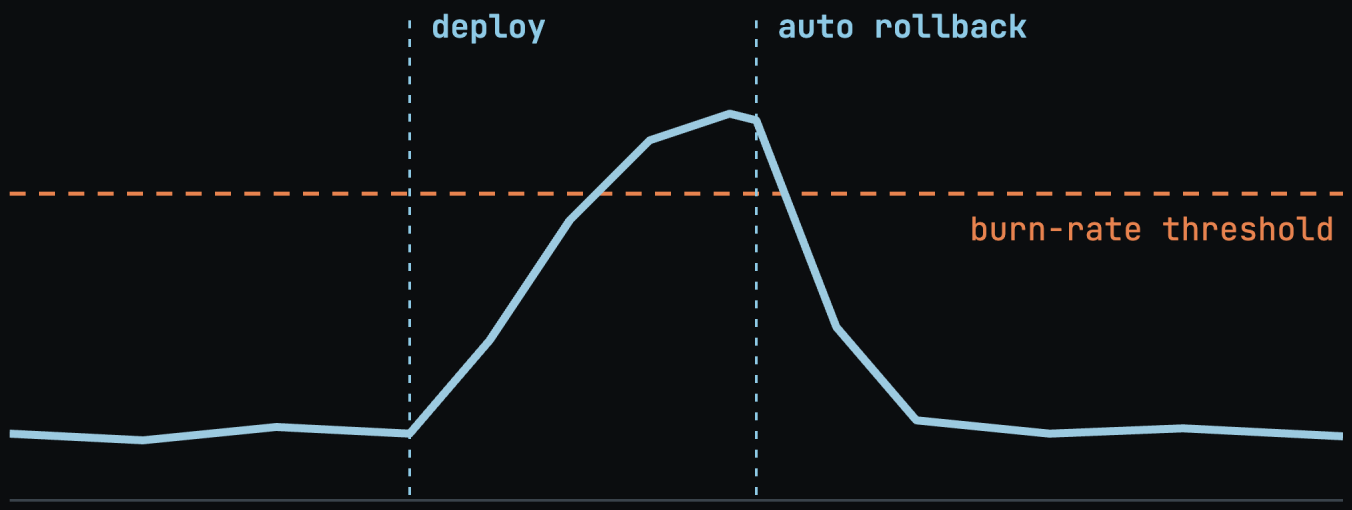
Progressive delivery by user segment

Slice by who the user is, not by a random percentage.



You control who takes the risk first.

Automated rollback on error budget burn



error rate over time

SLO · SERVICE-LEVEL OBJECTIVE

A health target written as a number, e.g. 99.9% of requests succeed.

Error budget: the 0.1% of failures you allow.

When a deploy eats that budget too fast, the system reverts it on its own.

Read: Dynatrace, service-level objectives documentation

Data safety

You can roll back code. You can't roll back data.

CODE

Bad deploy? Switch back to the old version and the problem is gone.

DATA

A deleted column, a double charge, a sent email: switching back undoes none of it.

THE SAFE WAY · RENAME THE COLUMN "name" TO "full_name" IN 3 STEPS

1 · ADD

`name` `full_name`

Add the new column and copy the data in. Old code keeps using name.

Safe to roll back

→

2 · SWITCH

`name` `full_name`

Deploy code that uses full_name. Broken? Roll back the code; name is still there.

Safe to roll back

→

3 · REMOVE

`full_name`

Weeks later, once everything is stable, delete the old name column.

No way back after this

AFTER MERGE · LAYER 4

05 Runtime safety and AI-powered ops

The three pillars

An AI ops agent is only as good as the signals it can read.

Logs

Timestamped events

```
12:04:11 user 412
login failed
```

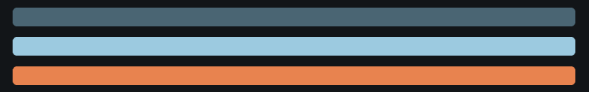
Metrics

Numbers over time

```
requests/s 1,240
p99 latency 480ms
```

Traces

One request across services



OpenTelemetry Vendor-neutral standard: instrument once, send anywhere, and make telemetry readable by an agent.

SLOs and error budgets



The budget is the unreliability you decided, out loud, is acceptable. (Bar not to scale.)

BUDGET TO SPARE

Ship features fast

BUDGET GONE

Stop and fix stability

REDUCING ALERT FATIGUE · EVERY PAGE SHOULD BE WORTH WAKING SOMEONE FOR

Symptom-based: "checkout is failing," not "CPU at 80%" · **Burn-rate:** how fast the budget is eaten · **Multiwindow:** catch spikes and slow rot

Read: Google SRE Workbook, Alerting on SLOs

Where AI changes the game

The “AI SRE”: an agent that reads your signals and starts investigating on its own.

DETECT

Anomaly detection

Learns normal daily and weekly rhythms; flags the strange.

Mature

COMPARE

Post-deploy analysis

Before vs. after signals; pins a regression on the PR.

Emerging

ANALYZE

Root-cause analysis

Gathers traces, deploys, errors; drafts a timeline. Human confirms.

Useful, moving fast

TRIAGE

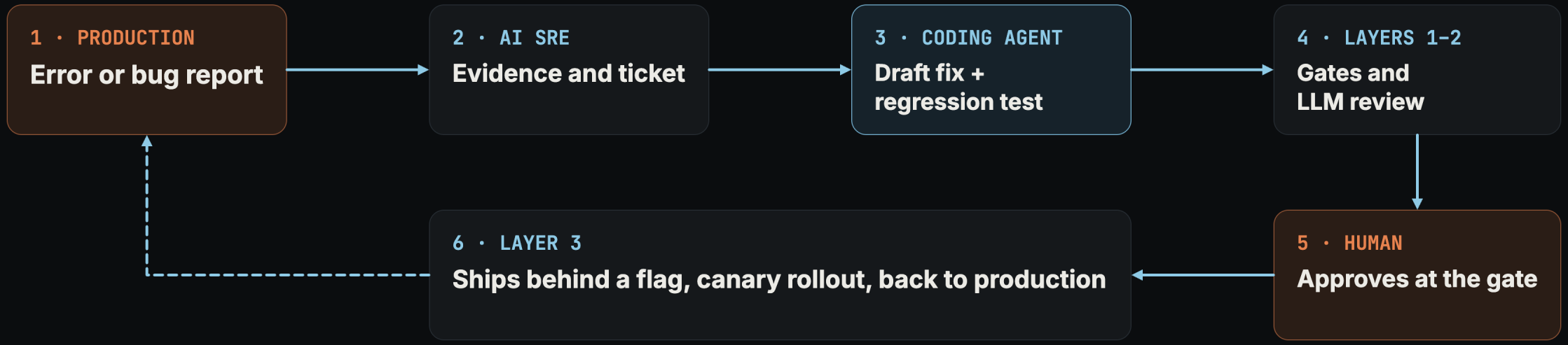
Bug-resolution pipeline

Dedupes errors, files tickets with repro steps, routes to owners.

Emerging

Real examples: Datadog Bits AI SRE · HolmesGPT (open source, CNCF Sandbox)

Closing the loop back to the coding agent



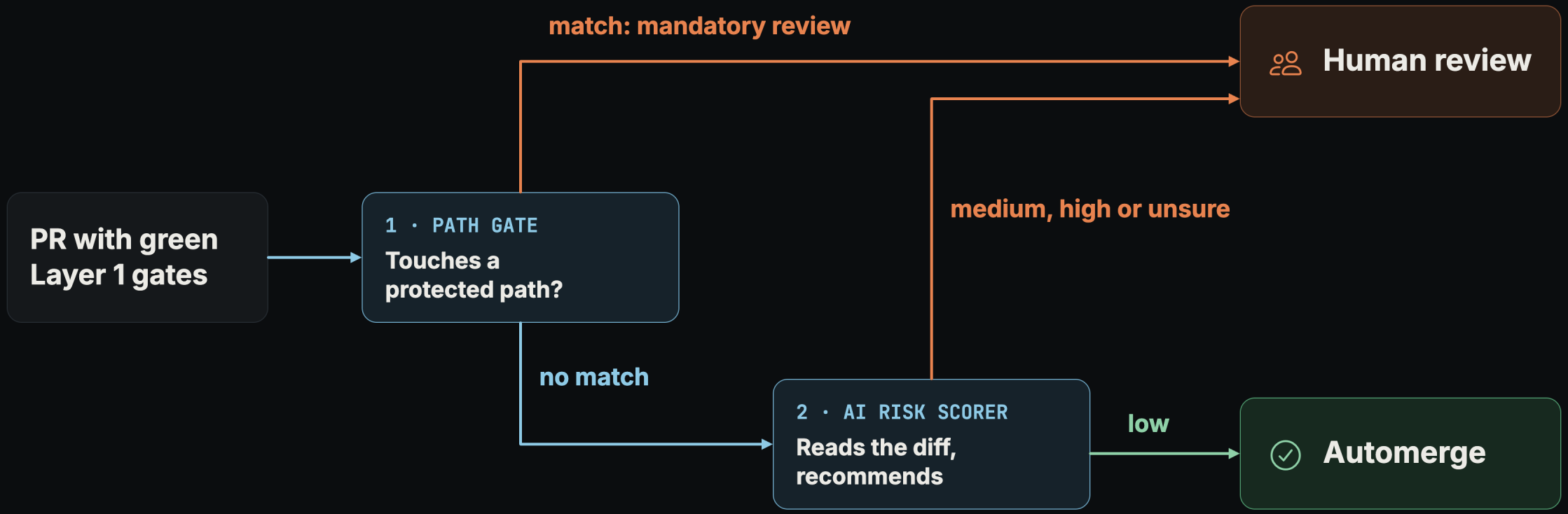
Every escape leaves a new test behind: it fails on the old code, passes on the fix, and guards the gate for good.

Shipping today: Sentry Seer · Dash0 Agent0 · Fully hands-off is still aspirational.

LAYER 5

06 The human in the loop

Two mechanisms decide what needs a human



The deterministic gate

Changes that always need a human. Boring, instant, impossible to argue with.

PATH PATTERNS

```
# Any change here needs a human.
services/payments/**
**/auth/**
**/*crypto*
**/migrations/**
infra/**
**/*.tf
.github/workflows/**
```

CONTENT PATTERNS

A scanner over changed lines: **DROP** or **DELETE** in SQL, a dangerous function, a credential shape, a deleted feature flag.

THE LIMIT

It only catches dangers you can name in advance. The scorer handles the rest.

The AI risk scorer

Using AI to decide when not to trust AI. Safe under one condition.

IT MUST FAIL SAFE

When unsure, round up to a human.

IT WEIGHS

Blast radius, reversibility, test coverage, diff size, hot or fragile files.

It can add a human. It can never override the gate.

OUTPUT (JSON)

```
{
  "risk": "medium",
  "recommendation": "one_human",
  "reviewer_domain": "payments",
  "what_could_go_wrong":
    "Retry may double-charge.",
  "reason": "New write path,
    no idempotency test."
}
```

How human review is changing

BEFORE

Line by line: the missing null check, the off-by-one

NOW

Intent and fit: is this the right thing to build?

ROUTE TO THE RIGHT HUMAN

```
# CODEOWNERS
services/payments/** @payments-team
```

A payments change for someone who never touched payments is review in name only.

REVIEWING THE REVIEWER

Log every override and feed it back into the rules.

Humans force reviews the scorer waved through: **too loose**.
Humans merge what it flagged: **crying wolf**.

07 Putting it together

A reference pipeline, end to end

Cheap checks run first and catch the most. Human attention is spent last.

SECONDS · ~FREE

- Local harness loop
- PR opens: Layer 1 gates

Run on everything, every time. Fail = stop.

MINUTES · CENTS

- LLM review, fresh session
- Path gate + risk scorer

Run on every real change.

HUMAN · RATIONED

- Domain reviewer, only if routed

One question: is this the right thing to build?

HOURS · ON PURPOSE

- Deploy behind a flag (off)
- Canary + SLO watch → 100%

Time lets production tell the truth while the blast radius is tiny.

What to adopt first

Build from the bottom up. Add the next layer when the pain says so.

STEP 1

Deterministic gates

Tests, types, lint, secrets. About a week.

STEP 2

LLM review

One checklist reviewer on a clean baseline.

STEP 3

Risk scoring and routing

Only once you trust the gates and reviewer.

STEP 4

AIOps

Needs instrumentation, SLOs, flags and a fix path first.

Metrics for the system itself

Watch them together. They trade against each other.

Metric	What it tells you	Healthy trend
PR cycle time	Opened to merged. Should fall as volume rises.	Down
Escape rate	Bugs reaching production per 100 PRs	Flat or down
Revert rate	Merged changes rolled back. Gates or scorer too loose?	Flat or down
False positive rate	Bot flags that were fine. The noise number.	Down
Human time per PR	Minutes of attention on a routine change	Down

THE TRAP Automerge everything and cycle time hits zero, while escapes and reverts spike.

Failure modes

Each one looks like success from across the room. Early signs:

Bot fatigue
Bot comments resolved without replies

Rubber stamping
A payments change approved in 12 seconds

Alert flood
Few pages lead to any action

Flag debt
A flag list that only grows

Ignored findings
Open security findings keep rising

AI overconfidence
Humans correct the scorer the same way, again and again

Failure is invisible until you measure for it.

The trust ladder

Authority is earned one reversible step at a time, never assumed.

RUNG 4 · DEFAULT: NO

Autonomous on critical paths

RUNG 3

Autonomous on low-risk paths

RUNG 2

Advisory: enforces rules you wrote

RUNG 1 · START HERE

Shadow mode: it suggests, a human decides

Move up when the metrics say it has earned it. Move down the moment they say it hasn't.

Takeaways

The bottleneck moved

Writing code is cheap. Verifying it is the constraint.

Cheap checks first

Deterministic gates catch the most and feed the agent's own loop.

Layers lean on each other

Review, safe deploys and runtime checks each cover what the last missed.

Humans judge intent

Is this even the right thing to build?

The human stays in the loop, just not in front of every diff.